

NAIL094 — Decision Procedures and SAT/SMT Solvers

Look-Ahead Solver

Zerina Ahmetovic

1 Task overview

The branching rule of the DPLL solver [1] is replaced by a look-ahead procedure. Three difference heuristics are implemented and compared, two additional reasoning techniques are added, and both rely on an eager clause-length data structure. Look-ahead is switched on with `dp11 -lookahead [-heuristic=crh|wbh|bsh] [input]`.

This is an addition to the previous DPLL algorithm [1], not a separate solver: run without `-lookahead` still branches in index order and behaves exactly as that report describes. The two sets of numbers below therefore come from one program, not two, and the branching rule is the only thing that differs between them.

The benchmarks are the same non-random SATLIB [5] families used before, pigeonhole [7] and All Interval Series [6], with one uniform random 3-SAT [8] instance for contrast.

2 Implementation

At every node each free variable is tentatively assigned in both polarities and propagated. A conflict in one polarity makes the opposite literal a failed literal, which is forced; a conflict in both makes the node unsatisfiable. Otherwise the reduction is measured and undone.

Difference heuristics. All three follow the definitions of Heule and van Maaren [4]. Let $\#_k(l)$ be the number of occurrences of literal l in clauses of size k in the input formula, and $w(l) = \sum_{k \geq 2} \gamma_k \cdot \#_k(l)$.

- CRH weighs each newly created clause by its size, using Kullmann's constants $\gamma_2 = 1$, $\gamma_3 = 0.2$, $\gamma_4 = 0.05$, $\gamma_5 = 0.01$, $\gamma_6 = 0.003$, and above that the regression $\gamma_k = 20.4514 \cdot 0.218673^k$ these were fitted to. Both families reach into that range — pigeonhole has one clause per pigeon of size n , All Interval Series n of size n and $n - 1$ of size $n - 1$ — but the weights there are vanishing: a clause of size 8 counts about a nine-thousandth of a new binary clause and one of size 12 about a four-millionth, so CRH is in effect blind to them.
- WBH sums $w(\neg a) + w(\neg b)$ over the newly created binary clauses $(a \vee b)$, with $\gamma_k = 5^{3-k}$.
- BSH multiplies instead of adding, $w(\neg a) \cdot w(\neg b)$, with $\gamma_k = 2^{3-k}$.

The exponents used here are the ones given in the task assignment [3]: $\gamma_k = 5^{3-k}$ for WBH and $\gamma_k = 2^{3-k}$ for BSH. The weights $w(l)$ are computed once from the input formula and not recomputed as the search descends, which satz does; that is a simplification of the chapter's scheme rather than a faithful copy of it.

The two directions are combined by the MixDiff function of the same chapter, $1024 \cdot L \cdot R + L + R$, where L and R are the values for the two polarities. The product rewards variables that constrain both ways and the sum only breaks ties. The more reduced polarity is explored first, being the smaller subproblem.

Additional reasoning. Two techniques are implemented. *Local learning*: when the look-ahead on x derives y , the binary clause $\neg x \vee y$ holds under the current partial assignment and is added; such clauses are propagated like any other and are dropped again when the trail shrinks below the point at which they were added. *Autarky reasoning*: if a look-ahead creates no new clause, every clause it touched came out satisfied, so the assignment is an autarky and can be committed without branching.

Eager data structure. Propagation keeps, per clause, the number of its literals that are true and the number that are not false. The second is the clause’s current length, which is exactly what the heuristics read. Watched literals deliberately do not track it, so the look-ahead solver uses this third engine instead; the cost is that backtracking has to undo the counters, where the other two engines undo nothing at all.

The engine can also be selected on its own with `-counters`, which keeps the counters but branches in index order. Although this way would not be the natural use of the solver, it is kept as it is the only configuration in which the data structure varies and the search does not, so it is what makes the cost of the counters measurable.

3 Experimental evaluation

The measurements were taken with g++ 13.3.0 under WSL2 (Ubuntu 24.04) at `-std=c++20 -O2`. Each instance was run three times and the fastest kept, the timer covers the search only. All configurations agree on every instance, and reported models were checked against the formula.

instance	vars	result	plain DPLL	counters	CRH	WBH	BSH	WBH bare
hole6	42	UNSAT	6 490	6 490	478	478	478	1 038
hole7	56	UNSAT	65 560	65 560	3 358	3 358	3 358	10 498
hole8	72	UNSAT	756 686	756 686	26 878	26 878	26 878	121 182
ais6	61	SAT	57	57	4	3	6	3
ais8	113	SAT	550	550	6	5	5	6
ais10	181	SAT	8 973	8 973	13	10	12	13
ais12	265	SAT	192 780	192 780	29	6	6	6
uf50-01	50	SAT	1 016	1 016	9	9	4	13

Table 1: Number of decisions, that is the size of the search tree. The counters column is the eager engine under plain branching, a control rather than a solver variant; its tree is identical to plain DPLL by construction. WBH bare is WBH with local learning and autarky reasoning switched off.

Table 1 shows the search tree collapsing. Against plain DPLL the reduction is a factor of 14 to 28 on pigeonhole and far larger on All Interval Series, where `ais12` falls from 192 780 decisions to 6 under WBH.

That gap is a difference in kind rather than degree, and it follows from the verdicts. Pigeonhole is unsatisfiable, so every branch has to be refuted and none can be skipped; a better branching variable shortens the refutation but cannot avoid doing one, which bounds how much any heuristic can win. All Interval Series is satisfiable, so a heuristic that keeps choosing productive

variables can descend almost directly to a model, and six decisions on `ais12` is what that looks like.

The counters column repeats plain DPLL on every row. The same index-order branching over the same formula must produce the same tree, so any difference in time between those two columns is the data structure and nothing else.

The three heuristics separate only where the instance gives them room to. On pigeonhole all three agree exactly, at 478, 3 358 and 26 878 decisions: the formula consists of binary clauses and one long clause per pigeon, the long clauses carry almost no weight at any of the three weightings, and the remaining binary structure is symmetric enough that every candidate ties. The difference appears on All Interval Series, where WBH and BSH find `ais12` in 6 decisions against CRH’s 29.

The single random instance in Table 1 settles nothing either way, so the three were also run over 300 instances of the SATLIB uniform random 3-SAT set, all 50 variables and 218 clauses. Look-ahead cuts the mean tree there from 418.2 decisions to about four, but it does not separate the heuristics: the means are 4.11, 4.07 and 4.30 for CRH, WBH and BSH, and every pairwise difference taken over the same instances has a 95% confidence interval straddling zero. The ordering is not even stable — over the first hundred instances BSH led, over three hundred it trails. The choice between the three shows up on structured instances and not on random ones.

The last column shows what the additional reasoning is worth (with local learning and autarky reasoning switched off), `hole8` needs 121 182 decisions instead of 26 878, more than four times as many.

instance	vars	result	plain DPLL	counters	CRH	WBH	BSH	WBH bare
hole6	42	UNSAT	0.0008	0.0016	0.0056	0.0061	0.0059	0.0066
hole7	56	UNSAT	0.0095	0.0122	0.0470	0.0518	0.0529	0.0751
hole8	72	UNSAT	0.1192	0.1611	0.4931	0.5440	0.5361	1.0118
ais6	61	SAT	0.0001	0.0001	0.0007	0.0008	0.0009	0.0005
ais8	113	SAT	0.0009	0.0010	0.0036	0.0027	0.0034	0.0022
ais10	181	SAT	0.0144	0.0157	0.0253	0.0139	0.0175	0.0088
ais12	265	SAT	0.3563	0.3707	0.1835	0.0204	0.0205	0.0115
uf50-01	50	SAT	0.0008	0.0011	0.0009	0.0008	0.0006	0.0008

Table 2: Running time in seconds for the same runs, fastest of three, search only.

Table 2 shows that a smaller tree is not always a faster solve. On pigeonhole look-ahead loses: `hole8` takes 0.4931s under CRH against plain DPLL’s 0.1192s, because probing every free variable at every node costs more than the 28-fold saving in nodes is worth. On All Interval Series it wins decisively, `ais12` dropping from 0.3563s to 0.0204s under WBH. Here the decision counts carry straight over into time: CRH needs 29 decisions to the others’ 6 and is correspondingly slower at 0.1835s. Two instances, `ais6` and `uf50-01`, run in under two milliseconds in every configuration, so their ratios are not worth reading.

Because the counters column shares plain DPLL’s tree, the slowdown on pigeonhole splits into two factors that multiply. On `hole8` the eager engine alone costs 0.1611s against 0.1192s, a factor of 1.35, and look-ahead on top of it costs 0.4931s against 0.1611s, a factor of 3.06; together they give the 4.14 observed. In time, look-ahead adds 0.374s to the plain solve, of which the counters are 0.042s and the probing 0.332s — so the bookkeeping is 11% of the excess and the probing the remaining 89%. On All Interval Series the engine is close to free, being a factor of 1.04 on `ais12`, and look-ahead still gains a factor of 18 on top of it, 0.3707s against 0.0204s.

The additional reasoning follows the same split. On pigeonhole it pays for itself, cutting `hole8`

from 1.0118 s to 0.5440 s. On All Interval Series the tree is already down to single-digit decisions, so there is nothing left for it to save and maintaining the learned clauses is a net cost: `ais12` is faster without it, at 0.0115 s against 0.0204 s.

Code

The full source is in the accompanying repository [2]. `src/look_ahead.hpp` holds the look-ahead procedure and the three heuristics, `src/eager_counters.hpp` the counter-based propagation and the learned clause store. Both include `src/dpll.hpp` and nothing in `dpll.hpp` refers back to them. The `-counters` flag selecting the engine under plain branching is in `src/dpll.cpp`. `tests/check_lookahead.py` asserts that every configuration agrees and validates the reported models, `tests/fuzz.py` checks each heuristic against exhaustive search on random instances, and `tests/bench_lookahead.py` produces the measurements.

Summary

Look-ahead buys a much smaller search tree at a large cost per node, and whether that is worth it follows the verdict. It wins by more than an order of magnitude on the satisfiable All Interval Series instances, where a good heuristic descends almost straight to a model; it loses on the unsatisfiable pigeonhole ones, where every branch must be refuted, the tree shrinks by only a factor of 28 and the solve still takes four times longer. Running the eager engine under plain branching splits that loss: on `hole8` the counters account for 11% of it and the probing for the rest.

The choice between the three heuristics matters far less than either. They are identical on pigeonhole, statistically indistinguishable over 300 random instances, and separate only on All Interval Series, while the additional reasoning makes a larger difference than any of them. No configuration measured here is best everywhere, and the ones that win do so for reasons visible in the formula rather than by being better in general.

References

- [1] Z. Ahmetovic. DPLL experimental evaluation, an earlier report of this series. Included in the accompanying repository.
- [2] Z. Ahmetovic. Source repository for the NAIL094 programming tasks. <https://github.com/inferno73/SAT-solver-NAIL094>
- [3] P. Kučera. NAIL094 programming task: Look-Ahead Solver. https://ktiml.mff.cuni.cz/~kucerap/satsmt/practical/task_look_ahead.php
- [4] M. Heule and H. van Maaren. Look-Ahead Based SAT Solvers. In *Handbook of Satisfiability*, chapter 5, pp. 155–184. Frontiers in Artificial Intelligence and Applications, vol. 185, IOS Press, 2009. https://www.cs.utexas.edu/~marijn/publications/p01c05_lah.pdf
- [5] H. H. Hoos and T. Stützle. SATLIB: An Online Resource for Research on SAT. <https://www.cs.ubc.ca/~hoos/SATLIB/benchm.html>
- [6] SATLIB benchmark problems, All Interval Series. <https://www.cs.ubc.ca/~hoos/SATLIB/Benchmarks/SAT/AIS/descr.html>
- [7] SATLIB benchmark problems, DIMACS pigeon hole. <https://www.cs.ubc.ca/~hoos/SATLIB/Benchmarks/SAT/DIMACS/PHOLE/descr.html>

- [8] SATLIB benchmark problems, Uniform Random-3-SAT. <https://www.cs.ubc.ca/~hoos/SATLIB/Benchmarks/SAT/RND3SAT/descr.html>